Software Theory Report
Audio360

Team #6, Six Sense
Omar Alam
Sathurshan Arulmohan
Nirmal Chaudhari
Kalp Shah
Jay Sharma

1 Revision History

Table 1: Revision History

Date	**Developer(s)**	**Change**
Mar 7, 2026	Kalp	Direction of Arrival theory and prototyping sections.
Apr 6, 2026	Nirmal	Introduction and Classification sections.
Apr 6, 2026	Sathurshan	Editing Document.

2 Symbols, Abbreviations and Acronyms

symbol	description
Audio360	360 Audio analysis system on smart glasses
DCT	Discrete Cosine Transform
DOA	Direction of Arrival
FFT	Fast Fourier Transform
FR	Functional Requirement
HUD	Heads Up Display
LDA	Linear Discriminant Analysis
M	Module
NFR	Non-Functional Requirement
PCA	Principal Component Analysis
R/Req	Requirement
SRS	Software Requirements Specification
SVM	Support Vector Machines
T	Test
UT	Unit Test
VnV	Verification and Validation

Table 2: Table of abbreviations and acronyms used in Audio360.

See SRS Documentation at Symbols, Abbreviations, and Acronyms for a complete table used in Audio360.

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Introduction **1**

4 Direction of Arrival **1**

4.1 DoA Theory . . . 1

4.1.1 GCC-PHAT (Generalized Cross-Correlation with Phase Transform) . 2

4.1.2 MUSIC (Multiple Signal Classification) . . . 3

4.1.3 SRP-PHAT (Steered Response Power with Phase Transform) . . . 3

4.1.4 FRIDA (FRI-Based DOA) . . . 4

4.2 DoA Python Prototyping . . . 4

4.2.1 Room Simulation with PyRoomAcoustics . . . 5

4.2.2 Test Data Generation . . . 6

4.2.3 Algorithm Testing and Comparison . . . 7

4.3 DoA Embedded Development . . . 8

5 Classification **9**

5.1 Dataset Preparation . . . 9

5.2 Experiment Approaches . . . 9

5.2.1 Support Vector Machines (SVM) . . . 9

5.2.2 Cosine Similarity . . . 10

5.2.3 Linear Discriminant Analysis (LDA) . . . 10

List of Tables

1 Revision History . . . i

2 Table of abbreviations and acronyms used in Audio360. . . . ii

List of Figures

1 Room layout used in PyRoomAcoustics simulation. . . . 5

2 Microphone responses after room simulation. . . . 6

3 Frequency-domain responses of microphone signals. . . . 7

4 DoA estimation results: comparison of MUSIC, SRP-PHAT, and FRIDA with ground truth. . . . 8

5 Visualization of audio samples in LDA space. . . . 11

3 Introduction

This document presents the prototyping efforts and theoretical foundations related to the software components of Audio360. It focuses on two key capabilities: direction of arrival (DoA) estimation and sound classification. It also describes the Python prototyping workflow used to validate algorithms, generate test data, and compare performance. This document was selected because it provides a comprehensive overview of how PyRoomAcoustics [1] was leveraged to simulate exactly how the system would perform in real conditions, which was crucial for validating the algorithms early on. It also provides detailed descriptions of the DoA and classification algorithms that were tested, and the results obtained from those tests, which informed the final design choices for the embedded implementation.

The structure of the document is as follows. The DoA section covers the theory and prototyping of DoA estimation, including detailed descriptions of the algorithms used (GCC-PHAT, MUSIC, SRP-PHAT, and FRIDA) and the PyRoomAcoustics simulation workflow. The Classification section discusses the sound classification experiments, including dataset preparation and the approaches tested (SVM, Cosine Similarity, and LDA). Each section includes theoretical background, implementation details, and results from the prototyping efforts.

4 Direction of Arrival

Direction of arrival (DoA) estimation is a fundamental capability of Audio360. Given synchronized multi-channel audio from a microphone array, the system must estimate the angular direction of sound sources on a 2D horizontal plane relative to the user (FR5.2, FR5.3). This section presents the theoretical foundations of the DoA algorithms used in the project and describes the Python prototyping workflow employed to validate and compare them.

4.1 DoA Theory

The DoA problem can be stated as follows. Let $\mathbf{x}_m(t)$ denote the signal received at microphone m at time t, for $m = 1, \ldots, M$ and $M \geq 2$. A sound source at position $\mathbf{p}_s$ emits a signal $s(t)$ that propagates through the medium. Due to the finite speed of sound c, the signal arrives at each microphone with a time delay $\tau_m = \|\mathbf{p}_m - \mathbf{p}_s\|/c$, where $\mathbf{p}_m$ is the position of microphone m. The received signal is a delayed, attenuated, and possibly reverberant version of the source. Estimating the direction of arrival amounts to inferring the direction vector from the array center toward the source from the observed multi-channel signals.

The project employs four DoA algorithms: Generalized Cross-Correlation with Phase Transform (GCC-PHAT), Multiple Signal Classification (MUSIC), Steered Response Power with Phase Transform (SRP-PHAT), and FRI-based DOA (FRIDA). Each algorithm is described below.

4.1.1 GCC-PHAT (Generalized Cross-Correlation with Phase Transform)

GCC-PHAT estimates the time difference of arrival (TDOA) between pairs of microphones. For a single source in the far field, the TDOA between microphones i and j is related to the source direction by the geometry of the array. Given TDOAs for multiple pairs, the source direction can be recovered via triangulation or closed-form solutions for specific array layouts [2].

Let $x_i(n)$ and $x_j(n)$ denote the discrete-time signals from microphones i and j, with sampling frequency f_s. The cross-correlation between them is

$$R_{ij}(\tau) = \mathbb{E}\left[x_i(n)\, x_j(n+\tau)\right]. \tag{1}$$

In the frequency domain, the cross-power spectral density is

$$G_{ij}(f) = X_i(f)\, \overline{X_j(f)}, \tag{2}$$

where $X_i(f)$ and $X_j(f)$ are the Fourier transforms of x_i and x_j, and $\bar{\cdot}$ denotes complex conjugation. The generalized cross-correlation (GCC) with a weighting function $\Psi(f)$ is

$$R_{ij}^{(\Psi)}(\tau) = \int_{-\infty}^{\infty} \Psi(f)\, G_{ij}(f)\, e^{j2\pi f\tau}\, df. \tag{3}$$

The Phase Transform (PHAT) weighting discards magnitude and retains only phase information:

$$\Psi_{\text{PHAT}}(f) = \frac{1}{|G_{ij}(f)| + \epsilon}, \tag{4}$$

where $\epsilon > 0$ is a small constant to avoid division by zero. This yields

$$R_{ij}^{\text{PHAT}}(\tau) = \int_{-\infty}^{\infty} \frac{G_{ij}(f)}{|G_{ij}(f)| + \epsilon}\, e^{j2\pi f\tau}\, df. \tag{5}$$

The TDOA estimate is the lag that maximizes the GCC-PHAT:

$$\hat{\tau}_{ij} = \frac{1}{f_s} \cdot \arg \max_{\ell \in [-\ell_{\max}, \ell_{\max}]} R_{ij}^{\text{PHAT}}(\ell), \tag{6}$$

where $\ell_{\max} = \lceil \tau_{\max} f_s \rceil$ and $\tau_{\max}$ is the maximum physically feasible delay given the inter-microphone distance. In practice, the integral is computed via the inverse FFT of the weighted cross-spectrum.

For a rectangular four-microphone array, pairs $(1, 2)$ and $(4, 3)$ provide horizontal-axis TDOAs; pairs $(3, 2)$ and $(4, 1)$ provide vertical-axis TDOAs. The normalized delays d_x and d_y are averaged over each axis and converted to an azimuth angle θ via $\theta = \arctan 2(d_y, d_x) - \pi/2$, with appropriate coordinate alignment for the glasses frame.

4.1.2 MUSIC (Multiple Signal Classification)

MUSIC is a subspace-based, high-resolution DoA algorithm [3]. It exploits the structure of the spatial covariance matrix to separate signal and noise subspaces.

The received signal vector at frequency f is modeled as

$$\mathbf{x}(f) = \mathbf{A}(\theta)\,\mathbf{s}(f) + \mathbf{n}(f), \tag{7}$$

where $\mathbf{A}(\theta) = [\mathbf{a}(\theta_1), \ldots, \mathbf{a}(\theta_D)]$ is the $M \times D$ steering matrix for D sources, $\mathbf{a}(\theta)$ is the steering vector for direction θ, $\mathbf{s}(f)$ is the $D \times 1$ source vector, and $\mathbf{n}(f)$ is additive noise. For a plane wave from direction θ, the steering vector has elements

$$a_m(\theta) = \exp\left(j\,\frac{2\pi f}{c}\,\mathbf{r}_m^\top \mathbf{u}(\theta)\right), \tag{8}$$

where $\mathbf{r}_m$ is the position of microphone m and $\mathbf{u}(\theta)$ is the unit direction vector.

The spatial covariance matrix is

$$\mathbf{R}_x = \mathbb{E}\left[\mathbf{x}\mathbf{x}^H\right] = \mathbf{A}\mathbf{R}_s\mathbf{A}^H + \sigma_n^2\mathbf{I}, \tag{9}$$

where $\mathbf{R}_s$ is the source covariance matrix and σ_n^2 is the noise power. Eigenvalue decomposition yields

$$\mathbf{R}_x = \mathbf{U}_s\mathbf{\Lambda}_s\mathbf{U}_s^H + \mathbf{U}_n\mathbf{\Lambda}_n\mathbf{U}_n^H. \tag{10}$$

The columns of $\mathbf{U}_s$ span the signal subspace (corresponding to the D largest eigenvalues); the columns of $\mathbf{U}_n$ span the noise subspace. The key property is that steering vectors for true source directions are orthogonal to the noise subspace: $\mathbf{a}^H(\theta)\mathbf{U}_n\mathbf{U}_n^H\mathbf{a}(\theta) = 0$ when θ is a true DoA.

The MUSIC spatial spectrum is

$$P_{\text{MUSIC}}(\theta) = \frac{1}{\mathbf{a}^H(\theta)\,\mathbf{U}_n\mathbf{U}_n^H\,\mathbf{a}(\theta)}. \tag{11}$$

DoA estimates are the D peaks of $P_{\text{MUSIC}}(\theta)$ over the search range. MUSIC requires knowing or estimating D and performs an eigendecomposition per frequency bin; it offers high angular resolution but is sensitive to model mismatch and coherent sources.

4.1.3 SRP-PHAT (Steered Response Power with Phase Transform)

SRP-PHAT is a beamforming-based method that searches over candidate source positions to maximize the output of a steered delay-and-sum beamformer [4]. It is robust to reverberation and noise because the PHAT weighting emphasizes phase coherence over magnitude.

For a candidate source position $\mathbf{x}$, the time difference of arrival between microphones m_1 and m_2 is

$$\tau_{m_1,m_2}(\mathbf{x}) = \frac{1}{c}\left(\|\mathbf{x} - \mathbf{p}_{m_1}\| - \|\mathbf{x} - \mathbf{p}_{m_2}\|\right). \tag{12}$$

The steered response power at $\mathbf{x}$ is the sum of generalized cross-correlations evaluated at the corresponding TDOAs:

$$P(\mathbf{x}) = \sum_{m_1=1}^{M} \sum_{m_2=1}^{M} R_{m_1,m_2}^{\text{PHAT}}(\tau_{m_1,m_2}(\mathbf{x})). \tag{13}$$

With PHAT weighting, each $R_{m_1,m_2}^{\text{PHAT}}$ uses only phase information, reducing sensitivity to spectral coloration and reverberation.

The source position estimate is

$$\hat{\mathbf{x}}_s = \arg\max_{\mathbf{x}\in\mathcal{G}} P(\mathbf{x}), \tag{14}$$

where $\mathcal{G}$ is a grid of candidate positions (e.g., on a sphere or circle for 2D azimuth). The DoA is then the direction from the array center to $\hat{\mathbf{x}}_s$. SRP-PHAT avoids eigendecomposition but requires a grid search; resolution is limited by grid spacing and computational cost.

4.1.4 FRIDA (FRI-Based DOA)

FRIDA leverages the finite rate of innovation (FRI) framework to estimate DoA without a grid search [5]. It is designed for arbitrary microphone array geometries and coherently combines multi-band information.

The central insight is that for any array layout, entries of the spatial covariance matrix can be linearly transformed into a uniformly sampled sum of sinusoids. The frequencies of these sinusoids encode the DoA. Specifically, for a given frequency bin, the covariance between microphones i and j depends on the phase difference $\phi_{ij}(\theta) = (2\pi f/c)\,(\mathbf{r}_i - \mathbf{r}_j)^\top \mathbf{u}(\theta)$. By arranging covariance entries in a suitable structure, one obtains a signal of the form $\sum_d \alpha_d e^{j\omega_d n}$, where ω_d is related to the d-th source direction. FRI sampling theory allows recovery of ω_d from a small number of samples, and thus recovery of the DoA, without exhaustive search.

FRIDA achieves state-of-the-art resolution at low signal-to-noise ratios by coherently combining information across frequency bands. It supports both 2D (azimuth only) and 3D (azimuth and elevation) localization and handles multiple sources. The algorithm is implemented in the pyroomacoustics library [1].

4.2 DoA Python Prototyping

Before implementing DoA on the embedded microcontroller, the team developed a Python prototyping pipeline to validate algorithms, compare their performance, and generate labeled test data. This pipeline uses PyRoomAcoustics [1] to simulate acoustic propagation in a 3D room and produces multi-channel recordings that mimic what the microphone array would capture in real conditions.

4.2.1 Room Simulation with PyRoomAcoustics

PyRoomAcoustics provides a `ShoeBox` room model: a rectangular enclosure with configurable dimensions and acoustic properties. The simulation computes room impulse responses (RIRs) between sources and microphones using the image source method, which models reflections from walls, floor, and ceiling. The received signal at each microphone is the convolution of the source signal with the corresponding RIR, plus optional diffuse noise.

The prototyping code (in `src/prototyping/pyroomacoustics_sim/`) creates a room with dimensions $10 \times 10 \times 3$ metres and sampling frequency $f_s = 8000$ Hz (or 16 kHz in the continuous simulation). Four microphones are placed in a square configuration approximating the glasses form factor:

- Microphone 1: $(4.95, 5.10, 1.75)$ m (front-left)
- Microphone 2: $(5.05, 5.10, 1.75)$ m (front-right)
- Microphone 3: $(4.95, 5.00, 1.75)$ m (back-left)
- Microphone 4: $(5.05, 5.00, 1.75)$ m (back-right)

The array spans approximately 10 cm in each horizontal dimension, consistent with the physical spacing of microphones on the smart glasses.

A sound source is placed at a known position (e.g., $(9, 9, 1)$ m or $(1, 6, 1)$ m). The source signal can be a sine wave, a pre-recorded audio file (e.g., `car_horn_8k.mp3`), or live microphone input. The `room.simulate()` method convolves the source signal with each RIR and produces the multi-channel output.

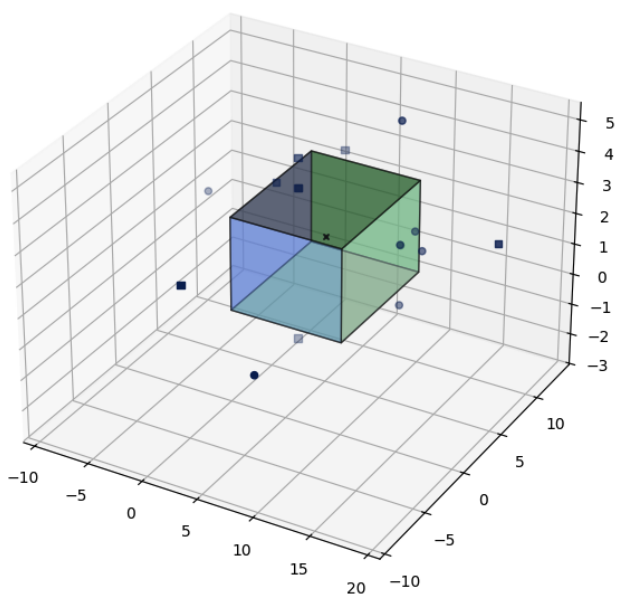

Figure 1: Room layout used in PyRoomAcoustics simulation.

4.2.2 Test Data Generation

After simulation, the multi-channel signals are saved as MP3 files (`mic_1.mp3`, `mic_2.mp3`, `mic_3.mp3`, `mic_4.mp3`) in a directory named for the test case (e.g., `example_mp3_audio_sources`). The `save_room_data()` function in `pyroom_helper.py` normalizes each channel to 16-bit integer range, wraps it in a `pydub` `AudioSegment`, and exports to MP3. A `README.md` is written alongside the audio files, documenting room dimensions, microphone positions, source positions, and sampling frequency. This metadata provides ground-truth DoA for validation.

The workflow is orchestrated by `example2.py`: create room, add microphones, load or generate source signal, add source, simulate, optionally plot the room and microphone responses, and save the data. The resulting files are "accurate" in the sense that they are produced by a physics-based simulation with known source and microphone geometry; the true azimuth can be computed as $\theta_{\text{true}} = \arctan 2(x_s - x_c, y_s - y_c)$ where (x_s, y_s) is the source position and (x_c, y_c) is the array center.

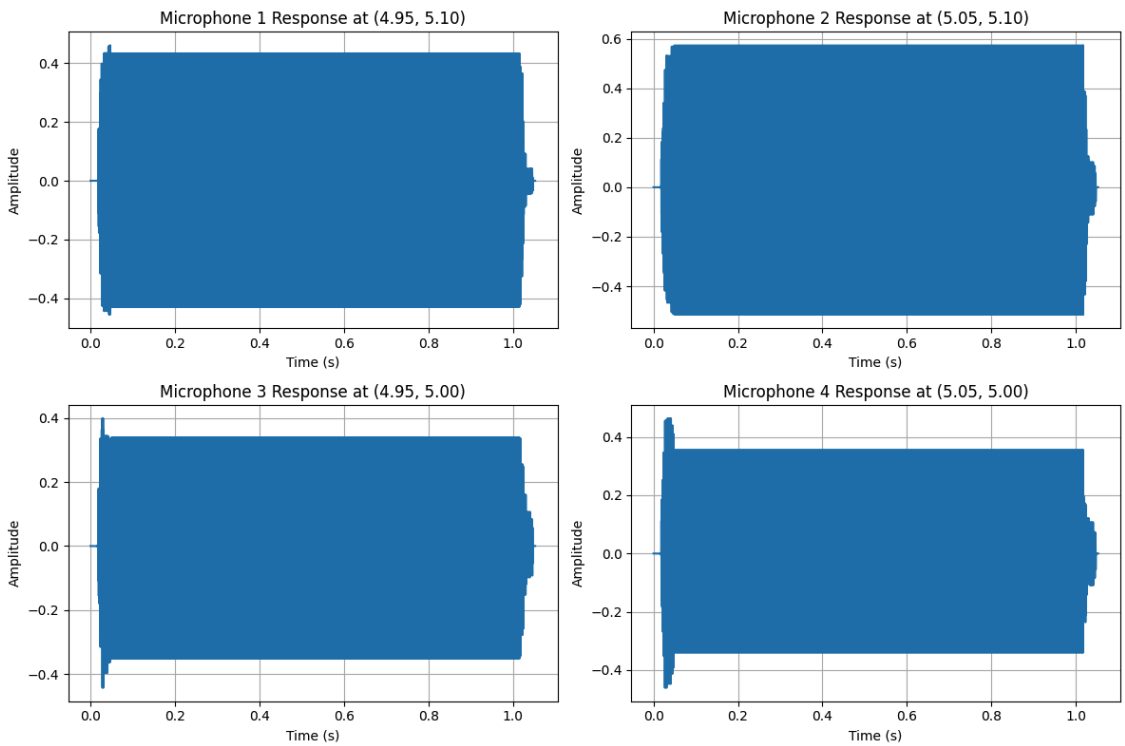

Figure 2: Microphone responses after room simulation.

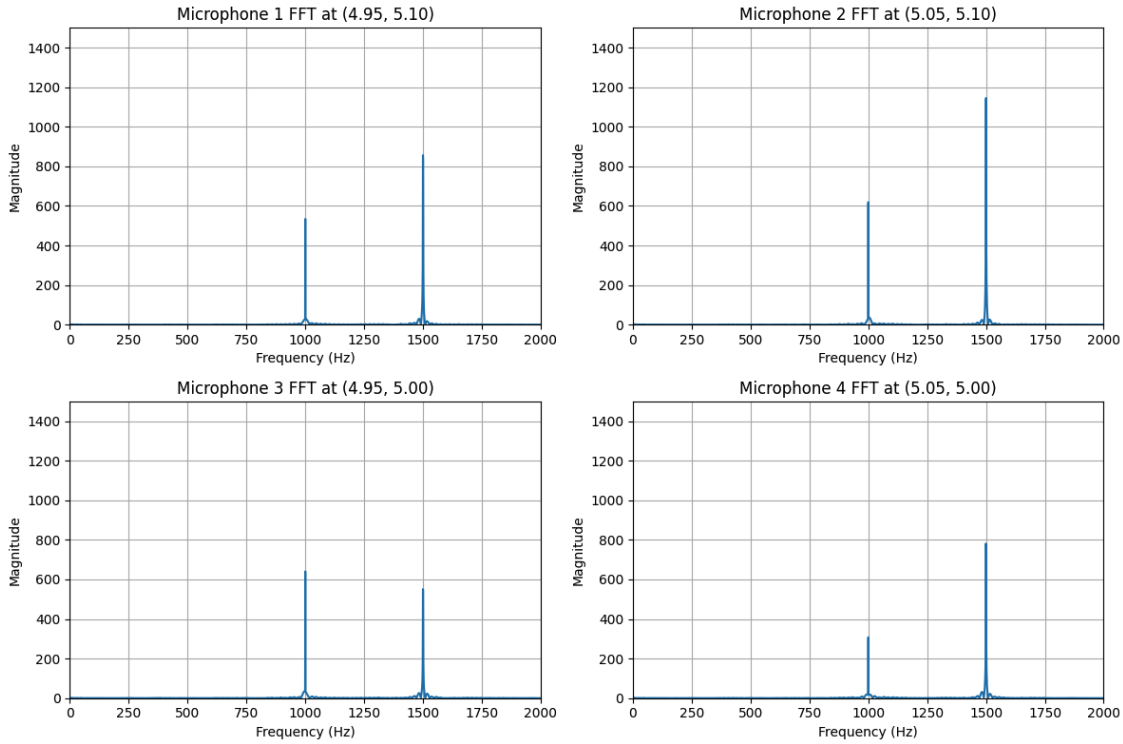

Figure 3: Frequency-domain responses of microphone signals.

4.2.3 Algorithm Testing and Comparison

The `doa_analysis.py` module loads the saved MP3 files, normalizes them to $[-1, 1]$, and converts the time-domain signals to the Short-Time Fourier Transform (STFT) format required by pyroomacoustics DoA algorithms. The STFT uses a Hanning window, FFT size $N_{\text{FFT}} = 256$, and hop size $N_{\text{FFT}}/2$. The frequency range is restricted to $[300, 3000]$ Hz to avoid DC and Nyquist aliasing and to focus on the most informative band for speech and environmental sounds.

Each algorithm is invoked with the same STFT data and microphone array geometry: `pra.doa.MUSIC`, `pra.doa.SRP`, and `pra.doa.FRIDA`. The estimated azimuth angles are stored in `doa_obj.azimuth_recon` (in radians). When a true source position is provided, the true azimuth is computed and compared to the estimates.

The `example_doa.py` script demonstrates the full pipeline: load data from `example_mp3_audio_sources` run MUSIC, SRP-PHAT, and FRIDA, and print a summary. The `visualize_doa_2d()` function produces a 2D polar plot showing microphone positions, estimated directions (colored arrows), and the true direction (black dashed arrow) when available.

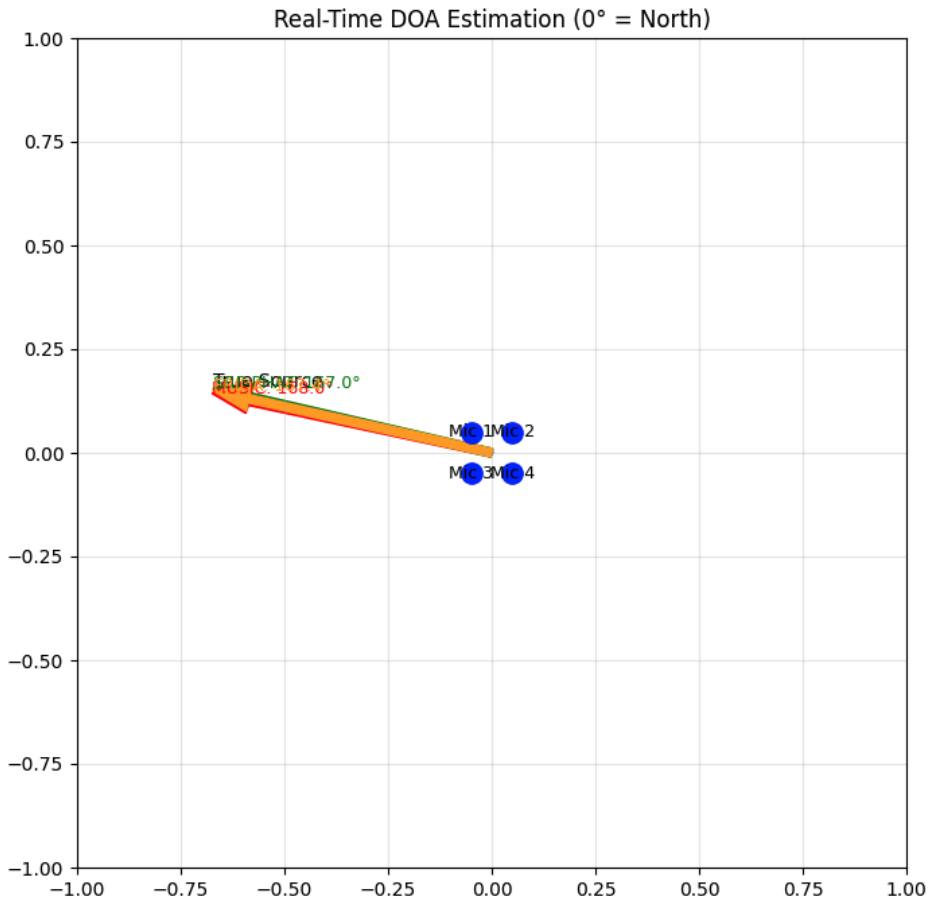

Figure 4: DoA estimation results: comparison of MUSIC, SRP-PHAT, and FRIDA with ground truth.

The continuous simulation (`run_simulation.py`) extends this workflow to live audio: a microphone captures real-time input, which is convolved with precomputed RIRs and fed to the DoA algorithms. The visualization updates continuously as new direction estimates are produced. This mode supports iterative tuning of algorithm parameters and validation under realistic conditions.

4.3 DoA Embedded Development

Once the proof of concept for DoA was successfully developed in Python, the team proceeded with confidence to migrate the algorithms into production source code in C and C++. The implementation is located in the `src/features/doa` directory. The GCC-PHAT algorithm was selected due to its lightweight computational requirements, making it well-suited for the target microcontroller compute abilities. Additionally, the implementation of DoA uses the command software design pattern, which allows the substitution of alternative DoA algorithms if the team decided to implement more.

Testing of the embedded implementation was conducted using pre-generated audio data. Audio samples for each microphone were created using PyRoomAcoustics and stored as mp3 files. These samples were then loaded into the test environment, where the DoA algorithm was executed. The computed angle was compared against the known ground truth, and assertions were used to validate the accuracy of the algorithm. The tests all pass within the angle accuracy threshold specified in requirement NFR5.3.

5 Classification

Classification of sounds is another key capability of Audio360 (FR5.1). The system must identify the type of sound source (e.g., siren, smoke_alarm, person_talking) from the multi-channel audio input provided in the PyRoom simulation. This section presents the theoretical experiments and prototyping efforts related to sound classification, and will give a high level breakdown of the algorithms and approaches used when testing different approaches. The source code for the embedded software is implemented in *src/features/classification*.

5.1 Dataset Preparation

The classification experiments relied on a dataset of labeled audio samples representing the target sound classes. The dataset was obtained from a combination of publicly available sound libraries such as FSD50k [6], and custom recordings of relevant sounds. Based on the desired classes, the final dataset was created using a balanced selection of samples for each class. The audio samples were also preprocessed to ensure consistency in sampling rate (16 kHz), duration (2 seconds), and format (mono).

5.2 Experiment Approaches

Several classification approaches were explored, including traditional machine learning methods like Support Vector Machines (SVM), and more light weight heuristic based approaches like Cosine Similarity and Linear Discriminant Analysis (LDA). All of these approaches were prototyped in Python and integrated with the continuous simulation script (run˙simulation.py) to evaluate their performance on the simulated multi-channel audio data. The following sections will give a high level overview of the approaches used and the results obtained from the experiments.

5.2.1 Support Vector Machines (SVM)

Support Vector Machines (SVM) are a popular supervised learning algorithm for classification tasks. Theoretically, SVMs work by finding the hyperplane that best separates the select classes in the feature space. In our experiments, we extracted 26 Mel-Frequency Cepstral Coefficients (MFCCs) per frame, then aggregated them over the 2 second duration by using the mean and standard deviation. For the purpose of testing and validating with this

model, we used a 80-20 train-test split on the dataset and performed 5-fold cross validation. We trained different SVM models with various kernel functions including Standard Scalar, Linear and RBF kernels. The performance of the SVM models seemed to be the best with the RBF kernels, achieving an accuracy of around 85% on the test set.

Although this approach was promising, we noticed there was a problem with efficiency when we tested the trained SVM model on the simulated multi-channel audio data. There was a noticeable delay in the classification outputs when testing this approach on continuous audio data, which is a concern for real time applications like Audio360, especially since in the real application, this model would be running on an embedded microcontroller with limited computational resources.

5.2.2 Cosine Similarity

Pivoting to approaches that used more heuristic based methods, we explored using Cosine Similarity for classification. This approach is based on the idea of representing each sound class as a vector in a high-dimensional feature space, and then classifying new audio samples by measuring the cosine similarity between the sample's feature vector and class vector. Here, we also used MFCCs as features for each targeted class, and created class vectors by averaging the MFCCs of all samples in each class.

When testing this approach on the simulated multi-channel audio data, we found that although this was a much more efficient approach compared to SVMs, the accuracy was significantly lower, achieving around a 45% accuracy on the test set. This suggests that while Cosine Similarity is computationally efficient, it may not capture the complex patterns in the audio data as effectively as SVMs.

5.2.3 Linear Discriminant Analysis (LDA)

After testing the SVM and Cosine Similarity approaches, we explored using explored using Linear Discriminant Analysis (LDA) for classification, which is a heuristic based method that seeks to find a linear combination of features that best separate the classes. Theoretically, it is similar to SVMs in that it seeks to find the hyperplane that best separates the classes, but it does so by maximizing the ratio of between-class to within-class variance, rather than maximizing the margins as in SVMs. In this approach there are two main steps involved after the MFCC feature extraction: Principle Component Analysis (PCA) and Linear Discriminant Analysis (LDA). PCA is used to reduce the dimensionality of the feature space by extracting the features that vary the most (considered the most important features). After PCA, LDA is applied to find the linear combination of PCA features that best separate the classes. The result of training are two matrices: the PCA projection matrix which is used to project the original features into the PCA space, and the LDA projection matrix which is used to project the PCA features into the LDA space. Evidently, although this approach is conceptually similar to SVMs, it is much more efficient to compute on continuous audio, since it is simply just matrix multiplications.

When testing this approach on the simulated multi-channel audio data, we found that

the accuracy was around 80% on the test set, which is slightly lower than the SVM approach, but significantly higher than the Cosine Similarity approach. This suggests that LDA is a promising approach for sound classification. And so, in our final prototype implementation, we used LDA for the classification.

Shown below is an image of three audio samples in the resulting LDA space, where we can see that the samples from the same class are clustered together, and the samples from different classes are separated.

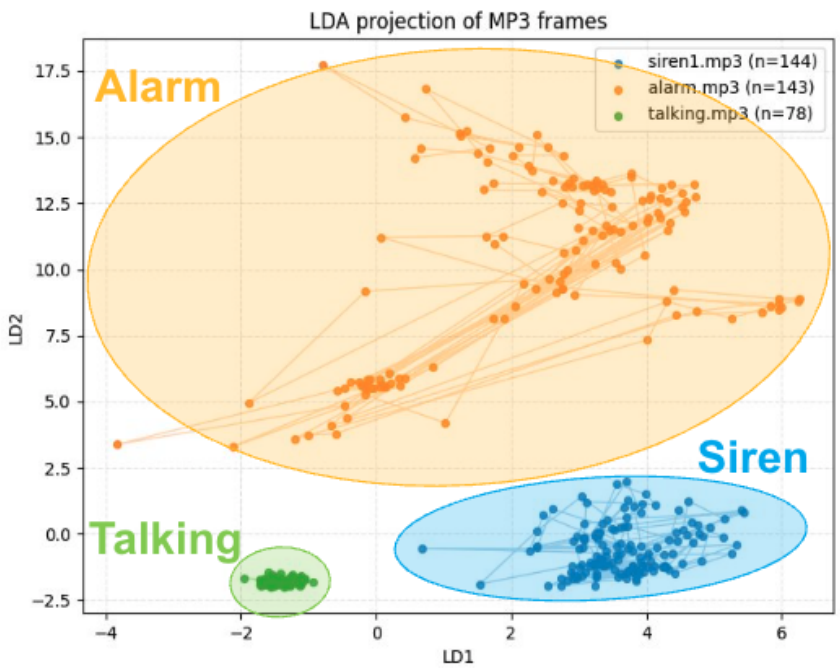

Figure 5: Visualization of audio samples in LDA space.

References

[1] fakufaku, "pyroomacoustics," 2025. [Online]. Available: https://pypi.org/project/pyroomacoustics/

[2] C. H. Knapp and G. C. Carter, "The generalized correlation method for estimation of time delay," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 4, pp. 320–327, 1976.

[3] R. O. Schmidt, "Multiple emitter location and signal parameter estimation," *IEEE Transactions on Antennas and Propagation*, vol. 34, no. 3, pp. 276–280, 1986.

[4] J. H. DiBiase, H. F. Silverman, and M. S. Brandstein, "Robust localization in reverberant rooms," in *Microphone Arrays: Signal Processing Techniques and Applications*. Springer, 2000, pp. 157–180.

[5] H. Pan, R. Scheibler, E. Bezzam, I. Dokmanić, and M. Vetterli, "FRIDA: FRI-based DOA estimation for arbitrary array layouts," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 3186–3190.

[6] FSD50k, "Fsd50k sound dataset," 2025. [Online]. Available: https://huggingface.co/datasets/Fhrozen/FSD50k